Practical 8: Working with Web GeoJSON

Day 4 — Duration: 1.5 Hours

SDI Training for ICT Professionals — Climate Risk Database

What you will accomplish: By the end of this practical, you will be able to understand the GeoJSON format, view GeoJSON data on the web, create and edit GeoJSON features using online tools, download GeoJSON layers from CRD via its API, convert between GeoJSON and other spatial formats, and embed GeoJSON maps in a simple web page.

Before you start: You need a modern web browser (Firefox or Chrome), an internet connection, and your CRD instance running at `http://localhost` with at least one vector layer uploaded.

Step 1: Understanding the GeoJSON Format

What is GeoJSON?

GeoJSON is an open standard format for encoding geographic data using JavaScript Object Notation (JSON). It is widely used on the web because:

- It is **human-readable** — you can open it in any text editor
- It is **lightweight** — smaller than Shapefiles for simple datasets
- It is **web-native** — all web mapping libraries support it natively
- It uses **WGS 84** (EPSG:4326) as its coordinate reference system

GeoJSON Structure

A GeoJSON file has a simple structure. Open a text editor and examine this example:

Example GeoJSON

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [39.2833, -6.8000]
      },
      "properties": {
        "name": "Dar es Salaam",
        "population": 4364541
      }
    }
  ]
}
```

Key concepts:

- **FeatureCollection** — the top-level container holding an array of features
- **Feature** — a single geographic object with geometry + properties
- **Geometry types:** Point, LineString, Polygon, MultiPoint, MultiLineString, MultiPolygon

- **Properties** — any attribute data (name, population, category, etc.)
- **Coordinates** — always in `[longitude, latitude]` order (not lat/lon!)

Step 1.1: Open a new tab in your web browser and go to: `https://geojson.org`

Step 1.2: Read the specification overview on the page. Note the supported geometry types.

Step 1.3: ☐ **Checkpoint:** Can you name at least four GeoJSON geometry types?

Step 2: Viewing GeoJSON on the Web with geojson.io

`geojson.io` is a free online tool for creating, viewing, and editing GeoJSON data. It provides an interactive map and a live JSON editor side by side.

Step 2.1: Open your browser and navigate to: `https://geojson.io`

Step 2.2: You will see a split screen: a **map** on the left and a **JSON editor** on the right.

Step 2.3: Copy the example GeoJSON from Step 1 above and paste it into the JSON editor panel (replace the existing content).

Step 2.4: The map should automatically zoom to show a point marker at Dar es Salaam.

Step 2.5: Click on the marker to see the properties popup showing the name and population.

Step 2.6: Try modifying the coordinates in the editor:

- Change `39.2833` to `39.2020` and `-6.8000` to `-6.1630`
- The marker should move to Zanzibar Stone Town

Step 2.7: ☐ **Checkpoint:** The marker is now displayed at the new location on the map.

Step 3: Creating GeoJSON Features with geojson.io

Step 3.1: On geojson.io, use the drawing tools in the **top-right** of the map:

- **Draw a marker** (point icon) — click on a location to place a point
- **Draw a line** (line icon) — click multiple points, then press Enter to finish
- **Draw a polygon** (polygon icon) — click to create vertices, click the first point to close
- **Draw a rectangle** (rectangle icon) — click and drag to create a bounding box

Step 3.2: Draw at least **three features** on the map:

- One **Point** marking a school or hospital you know
- One **Polygon** outlining a neighbourhood or park
- One **LineString** tracing a road or river

Step 3.3: After drawing each feature, click on it to add properties. In the popup, click **Add row** and add:

- `name` — a descriptive name
- `category` — e.g., "education", "residential", "transport"

Step 3.4: Watch how the JSON editor on the right updates automatically as you draw and add properties.

Step 3.5: Click **Save** → **GeoJSON** to download your file. Name it `my_features.geojson`.

Step 3.6: ☐ **Checkpoint:** You have a downloaded `my_features.geojson` file with at least 3 features.

Step 4: Downloading GeoJSON from CRD (GeoNode API)

The Climate Risk Database (CRD) platform built on GeoNode provides a WFS (Web Feature Service) endpoint that can return vector layers as GeoJSON.

Step 4.1: Open your CRD in a browser at `http://localhost`.

Step 4.2: Navigate to the **Datasets** page and click on any **vector** layer (e.g., a point or polygon dataset you uploaded earlier).

Step 4.3: Note the layer name shown on the page (e.g., `geonode:mwanza_businesses`).

Step 4.4: To download this layer as GeoJSON, construct the following URL in your browser address bar:

WFS GeoJSON Request

```
http://localhost/geoserver/ows?
  service=WFS&
  version=2.0.0&
  request=GetFeature&
  typeNames=geonode:YOUR_LAYER_NAME&
  outputFormat=application/json
```

Replace `YOUR_LAYER_NAME` with the actual layer name.

Step 4.5: The browser will display or download a GeoJSON response. If displayed in the browser:

- Right-click → **Save As** → save it as `crd_layer.geojson`

Step 4.6: Open the downloaded file in geojson.io by dragging and dropping it onto the map area. Verify that the features display correctly.

> **Tip:** If the layer name contains special characters or spaces, try URL-encoding them. For most CRD layers, the name is lowercase with underscores.

Step 4.7: You can also limit the number of features returned by adding `&count=10` to the URL to retrieve only the first 10 features.

Step 4.8: ☐ **Checkpoint:** You have downloaded a GeoJSON file from your CRD and viewed it on geojson.io.

Step 5: Converting Between GeoJSON and Other Formats

Method A: Using QGIS (GUI)

Step 5A.1: Open **QGIS** and drag your `crd_layer.geojson` file onto the map canvas.

Step 5A.2: Right-click the layer in the Layers panel → **Export** → **Save Features As...**

Step 5A.3: In the dialog:

- **Format:** Choose the target format (e.g., `ESRI Shapefile`, `GeoPackage`, `KML`)
- **File name:** Click Browse and choose a save location
- **CRS:** Keep as EPSG:4326 or change if needed

Step 5A.4: Click **OK** to export. The converted file is now saved.

Step 5A.5: To convert *to* GeoJSON: repeat the same steps but choose **GeoJSON** as the output format.

Step 5A.6: ☐ **Checkpoint:** You converted a GeoJSON file to another format using QGIS.

Method B: Using the Terminal (ogr2ogr)

For batch conversions or automation, use the `ogr2ogr` command-line tool (included with QGIS/GDAL):

Step 5B.1: Open a terminal. Convert a Shapefile to GeoJSON:

Shapefile to GeoJSON

```
ogr2ogr -f "GeoJSON" output.geojson input.shp
```

Step 5B.2: Convert GeoJSON to GeoPackage:

GeoJSON to GeoPackage

```
ogr2ogr -f "GPKG" output.gpkg input.geojson
```

Step 5B.3: Convert GeoJSON to KML:

GeoJSON to KML

```
ogr2ogr -f "KML" output.kml input.geojson
```

Step 5B.4: ☐ **Checkpoint:** You converted a spatial file using a terminal command.

Step 6: Exploring the CRD GeoJSON API Endpoints

CRD (GeoNode) provides a REST API that returns GeoJSON data. This is useful for integrating CRD data into web applications and dashboards.

Step 6.1: Open your browser and go to the GeoNode API datasets list:

CRD API Datasets Endpoint

```
http://localhost/api/v2/datasets/
```

Step 6.2: The response is JSON. Look for the `links` section of any dataset entry. Find links with `mime: application/json` — these are GeoJSON download links.

Step 6.3: Try the direct GeoJSON link for a dataset. The URL pattern is typically:

Direct GeoJSON Download

```
http://localhost/api/v2/datasets/DATASET_ID/download/
  ?export_format=json
```

Step 6.4: You can also filter features using CQL filters in WFS:

WFS with CQL Filter

```
http://localhost/geoserver/ows?
  service=WFS&version=2.0.0&
  request=GetFeature&
  typeNames=geonode:YOUR_LAYER&
  outputFormat=application/json&
  CQL_FILTER=category='residential'
```

Step 6.5: ☐ **Checkpoint:** You retrieved filtered GeoJSON data from CRD using the API.

Step 7: Displaying GeoJSON on a Web Map (Leaflet)

In this step, you will create a simple HTML file that displays your GeoJSON data on an interactive web map using the Leaflet library.

Step 7.1: Open a text editor (e.g., Mousepad, VS Code, or Notepad) and create a new file called `my_map.html`.

Step 7.2: Copy and paste the following code into the file:

my_map.html — Web Map with GeoJSON

```
<!DOCTYPE html>
<html>
<head>
  <title>My GeoJSON Map</title>
  <meta charset="utf-8" />
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet"
    href="https://unpkg.com/leaflet@1.9/dist/leaflet.css"/>
  <script
    src="https://unpkg.com/leaflet@1.9/dist/leaflet.js">
  </script>
  <style>
    #map { height: 100vh; width: 100%; }
  </style>
</head>
<body>
  <div id="map"></div>
  <script>
    // Create the map centred on Tanzania
    var map = L.map('map').setView([-6.5, 37.5], 6);

    // Add OpenStreetMap base layer
    L.tileLayer(
      'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
      { attribution: '&copy; OpenStreetMap contributors' }
    ).addTo(map);

    // Your GeoJSON data (inline)
    var geojsonData = {
      "type": "FeatureCollection",
      "features": [
        {
          "type": "Feature",
          "geometry": {
            "type": "Point",
            "coordinates": [39.2833, -6.8000]
          },
          "properties": {
            "name": "Dar es Salaam",
            "category": "city"
          }
        },
        {
          "type": "Feature",
          "geometry": {
            "type": "Point",
            "coordinates": [39.2020, -6.1630]
```

```
        },
        "properties": {
          "name": "Zanzibar",
          "category": "city"
        }
      }
    ]
  };

  // Add GeoJSON to map with popups
  L.geoJSON(geojsonData, {
    onEachFeature: function(feature, layer) {
      if (feature.properties) {
        var popup = '<b>' + feature.properties.name
          + '</b><br>Category: '
          + feature.properties.category;
        layer.bindPopup(popup);
      }
    }
  }).addTo(map);
</script>
</body>
</html>
```

Step 7.3: Save the file and open it in your web browser (double-click the file or drag it into the browser).

Step 7.4: You should see an interactive map with two markers. Click a marker to see its popup.

Step 7.5: ☐ **Checkpoint:** Your web page displays an interactive map with GeoJSON markers.

Step 8: Loading External GeoJSON Files

Instead of embedding GeoJSON data inline, you can load it from a file or a URL.

Step 8.1: To load GeoJSON from your CRD instance, modify the script section of `my_map.html`. Replace the inline `geojsonData` variable and the `L.geoJSON` block with:

Loading GeoJSON from CRD via fetch

```
<script>
  var map = L.map('map').setView([-6.5, 37.5], 6);

  L.tileLayer(
    'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
    { attribution: '&copy; OpenStreetMap contributors' }
  ).addTo(map);

  // Fetch GeoJSON from CRD WFS endpoint
  var wfsUrl = 'http://localhost/geoserver/ows?'
    + 'service=WFS&version=2.0.0'
    + '&request=GetFeature'
    + '&typeNames=geonode:YOUR_LAYER_NAME'
    + '&outputFormat=application/json';

  fetch(wfsUrl)
    .then(function(response) {
      return response.json();
    })
    .then(function(data) {
```

```
    var geojsonLayer = L.geoJSON(data, {
      onEachFeature: function(feature, layer) {
        var popup = '';
        for (var key in feature.properties) {
          popup += '<b>' + key + ':</b> '
            + feature.properties[key] + '<br>';
        }
        layer.bindPopup(popup);
      }
    }).addTo(map);
    // Zoom to the data extent
    map.fitBounds(geojsonLayer.getBounds());
  });
</script>
```

Step 8.2: Replace `YOUR_LAYER_NAME` with the name of a vector layer from your CRD.

Step 8.3: Save and refresh the browser. The map should load and display your CRD layer data.

> **Note on CORS:** If loading from a remote CRD, the browser may block the request due to Cross-Origin Resource Sharing (CORS) restrictions. For local development (`localhost`), this usually works fine.

Step 8.4: ☐ **Checkpoint:** Your web map loads GeoJSON data dynamically from the CRD WFS endpoint.

Step 9: Styling GeoJSON Features on the Map

Step 9.1: You can style GeoJSON features by adding a `style` function. Add this inside the `L.geoJSON` options:

Styling GeoJSON Features

```
var geojsonLayer = L.geoJSON(data, {
  style: function(feature) {
    // Style polygons and lines
    return {
      color: '#007064', // border colour
      weight: 2, // border width
      fillColor: '#D5F0EC', // fill colour
      fillOpacity: 0.5 // fill transparency
    };
  },
  pointToLayer: function(feature, latlng) {
    // Style points as circle markers
    return L.circleMarker(latlng, {
      radius: 8,
      fillColor: '#E6A800',
      color: '#00463C',
      weight: 2,
      fillOpacity: 0.8
    });
  },
  onEachFeature: function(feature, layer) {
    var popup = '';
    for (var key in feature.properties) {
      popup += '<b>' + key + ':</b> '
        + feature.properties[key] + '<br>';
```

```
    }
    layer.bindPopup(popup);
  }
}).addTo(map);
```

Step 9.2: Save and refresh. Polygons should now appear with green borders and light green fill. Points appear as amber circle markers.

Step 9.3: Try changing the colour values to match your preferences. The colours above use the Resilience Academy brand colours.

Step 9.4: ☐ **Checkpoint:** Your GeoJSON features are displayed with custom styles.

Step 10: Validating GeoJSON Data

Before sharing or uploading GeoJSON, it is good practice to validate it.

Step 10.1: Open your browser and go to: `https://geojsonlint.com`

Step 10.2: Paste your GeoJSON content into the text area and click **Validate**.

Step 10.3: The tool will report whether your GeoJSON is valid or show errors that need fixing.

Step 10.4: Common issues to watch for:

- **Coordinate order:** Must be `[longitude, latitude]`, not `[latitude, longitude]`
- **Unclosed polygons:** The first and last coordinate of a polygon ring must be identical
- **Missing commas:** Between features in the features array
- **Wrong type values:** Geometry type names are case-sensitive

Step 10.5: ☐ **Checkpoint:** Your GeoJSON file passes validation without errors.

Summary and Key Takeaways

What you learned today:

1. **GeoJSON format** — a lightweight, human-readable standard for web-based spatial data
2. **geojson.io** — an online tool for viewing, creating, and editing GeoJSON interactively
3. **CRD WFS endpoint** — how to download vector layers as GeoJSON from the Climate Risk Database
4. **Format conversion** — converting between GeoJSON and Shapefile/GeoPackage/KML using QGIS or ogr2ogr
5. **CRD REST API** — accessing GeoJSON data programmatically via the GeoNode API
6. **Leaflet web maps** — displaying GeoJSON data on interactive web maps using HTML and JavaScript
7. **Styling and popups** — customising the visual appearance of GeoJSON features on a map
8. **Validation** — checking GeoJSON for errors before sharing or uploading

Next steps: Try uploading the GeoJSON file you created in geojson.io back into CRD. Use the same data upload process you learned in Practical 4. This completes the full cycle: CRD →

GeoJSON → Web Map → CRD.